SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK B4 — Final Engineering Decision (5 Marks)

Task / Problem Statement

The district now connects 25 hospitals with 100× the present number of donors, and wants a nightly report of donors who will be eligible to donate within the next week. Present size is taken as 50,000 donors (the figure in A4), so the new size is **5,000,000 donors**.

Aim

To implement the report with the growable donor list, measure its time and memory, prove memory safety, and decide how the production system should store the donors.

Concept / Theory

Eligible next week means: age 18–65 and at least 90 days between the last donation and the date seven days from now. Three strategies are compared: **(1)** load all donors into a growable array (doubling realloc), **(2)** read the file in batches of 10,000 records and keep nothing, **(3)** keep a separate sorted index file of (ID, byte offset) and look donors up by binary search with fseek. The report program stores only the fields it needs (id, group, age, last-donation day) in a 16-byte record.

Procedure / Algorithm

1. Generate a text donor file of 50,000 and of 5,000,000 donors (246 MB).
2. Complete TODO 1 of the given growable list and run it.
3. Strategy 1: read the file line by line, add_donor for every line, then scan the list, write eligible donors to nightly_report.txt; measure time (clock_gettime) and peak memory (getrusage maximum resident set).
4. Strategy 2: same report in batches of 10,000, with a single fixed batch array.
5. Strategy 3: build the (ID, offset) index file, then time 20,000 random lookups by binary search with fseek; also time 100,000 lookups in the in-memory list.
6. Run valgrind and AddressSanitizer; compare the two reports (identical); decide.

Program / Source Code

Program 1 – the given growable list with TODO 1 completed (b4_todo.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;

/* Appends a donor, doubling the capacity when the list is full.
   Returns 1 on success, 0 if memory could not be allocated. */
int add_donor(DonorList *list, int id, const char *group)
{
    if (list->size == list->capacity) {
        int newcap = (list->capacity == 0) ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, (size_t)newcap * sizeof(Donor));
        if (p == NULL) return 0;          /* list->data is untouched and still valid */
        list->data = p;
        list->capacity = newcap;
    }
    list->data[list->size].id = id;
    strncpy(list->data[list->size].group, group, 3);
    list->data[list->size].group[3] = '\0';
    list->size++;
    return 1;
}

int main(void)
{
    DonorList list = {NULL, 0, 0};
```

```
    const char *groups[] = {"O+", "A+", "B-", "AB+", "O-"};
    for (int i = 0; i < 5; i++) {
        add_donor(&list, 7001 + i, groups[i]);
        printf("added %d (%s): size = %d, capacity = %d\n",
               list.data[i].id, list.data[i].group, list.size, list.capacity);
    }
    free(list.data);
    return 0;
}
```

Program 2 – nightly report and strategy comparison (b4_report.c), compiled with gcc -O2 -o b4_report b4_report.c. Usage: ./b4_report gen donors_5m.txt 5000000, ./b4_report load donors_5m.txt, ./b4_report batch donors_5m.txt, ./b4_report index donors_5m.txt donors_5m.idx:

```
#define _POSIX_C_SOURCE 200809L
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/resource.h>

/* compact donor record used by the nightly report */
typedef struct { int id; char group[4]; int age; int last_day; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;

#define MIN_GAP 90
static long TODAY;

static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv_sec + t.tv_nsec /
1e9; }
static long maxrss_kb(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_maxrss; }
static long days_from_date(const char *s)
{
    struct tm t = {0};
    if (sscanf(s, "%d-%d-%d", &t.tm_year, &t.tm_mon, &t.tm_mday) != 3) return -1;
    t.tm_year -= 1900; t.tm_mon -= 1;
    return (long)(timegm(&t) / 86400);
}
static int eligible_next_week(const Donor *d)
{
    return d->age >= 18 && d->age <= 65 && (TODAY + 7) - d->last_day >= MIN_GAP;
}

/* Appends a donor, doubling the capacity when the list is full.
   Returns 1 on success, 0 if memory could not be allocated. */
int add_donor(DonorList *list, int id, const char *group, int age, int last_day)
{
    if (list->size == list->capacity) {
        int newcap = list->capacity == 0 ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, (size_t)newcap * sizeof(Donor));
        if (p == NULL) return 0;              /* list->data is still valid, nothing lost */
        list->data = p;
        list->capacity = newcap;
    }
    list->data[list->size].id = id;
    strncpy(list->data[list->size].group, group, 3);
    list->data[list->size].group[3] = '\0';
    list->data[list->size].age = age;
    list->data[list->size].last_day = last_day;
    list->size++;
```

```
    return 1;
}

static int parse_line(const char *line, int *id, char *grp, int *age, long *day)
{
    char date[12];
    if (sscanf(line, "%d,%*[^,],%3[^,],%d,%*[^,],%10s", id, grp, age, date) != 4) return 0;
    *day = days_from_date(date);
    return *day >= 0;
}

static void generate(const char *path, long n)
{
    static const char *G[8] = {"A+","A-","B+","B-","AB+","AB-","O+","O-"};
    FILE *f = fopen(path, "w"); if (!f) { perror(path); exit(1); }
    unsigned long long x = 88172645463325252ULL;
    for (long i = 0; i < n; i++) {
        x ^= x << 13; x ^= x >> 7; x ^= x << 17;
        int age = 17 + (int)(x % 54);                   /* 17..70 */
        long day = TODAY - (long)((x >> 20) % 200);     /* last donation 0..199 days ago */
        time_t t = (time_t)day * 86400; struct tm *g = gmtime(&t);
        fprintf(f, "%ld,Donor%07ld,%s,%d,98%08ld,%04d-%02d-%02d\n", 1000000 + i, i, G[(x >> 8) % 8], age,
                i % 100000000, g->tm_year + 1900, g->tm_mon + 1, g->tm_mday);
    }
    fclose(f);
}

int main(int argc, char **argv)
{
    TODAY = days_from_date("2026-11-13");
    if (argc < 3) { fprintf(stderr, "usage: b4_report gen <file> <n> | load <file> | batch <file> | index <file> <idx>\n");
return 1; }
    const char *mode = argv[1], *path = argv[2];

    if (!strcmp(mode, "gen")) { generate(path, atol(argv[3])); return 0; }

    if (!strcmp(mode, "load")) {                /* strategy 1: everything in memory */
        double t0 = now();
        FILE *fp = fopen(path, "r");
        if (!fp) { perror(path); return 1; }
        DonorList list = {NULL, 0, 0};
        char line[256];
        while (fgets(line, sizeof line, fp)) {
            int id, age; char g[4]; long day;
            if (!parse_line(line, &id, g, &age, &day)) continue;
            if (!add_donor(&list, id, g, age, (int)day)) { perror("add_donor"); fclose(fp); free(list.data); return 1; }
        }
        fclose(fp);
        double t1 = now();
        FILE *out = fopen("nightly_report.txt", "w");
        if (!out) { perror("nightly_report.txt"); free(list.data); return 1; }
        long eligible = 0;
        for (int i = 0; i < list.size; i++)
            if (eligible_next_week(&list.data[i])) { fprintf(out, "%d %s\n", list.data[i].id, list.data[i].group); eligible++; }
        fclose(out);
        double t2 = now();
        printf("strategy: ALL IN MEMORY\n");
        printf("donors loaded      : %d (capacity %d, %zu bytes per record)\n", list.size, list.capacity, sizeof(Donor));
        printf("list memory in use : %.1f MB used, %.1f MB allocated\n", list.size * sizeof(Donor) / 1048576.0,
list.capacity * sizeof(Donor) / 1048576.0);
        printf("eligible next week : %ld\n", eligible);
        printf("load time          : %.3f s\n", t1 - t0);
```

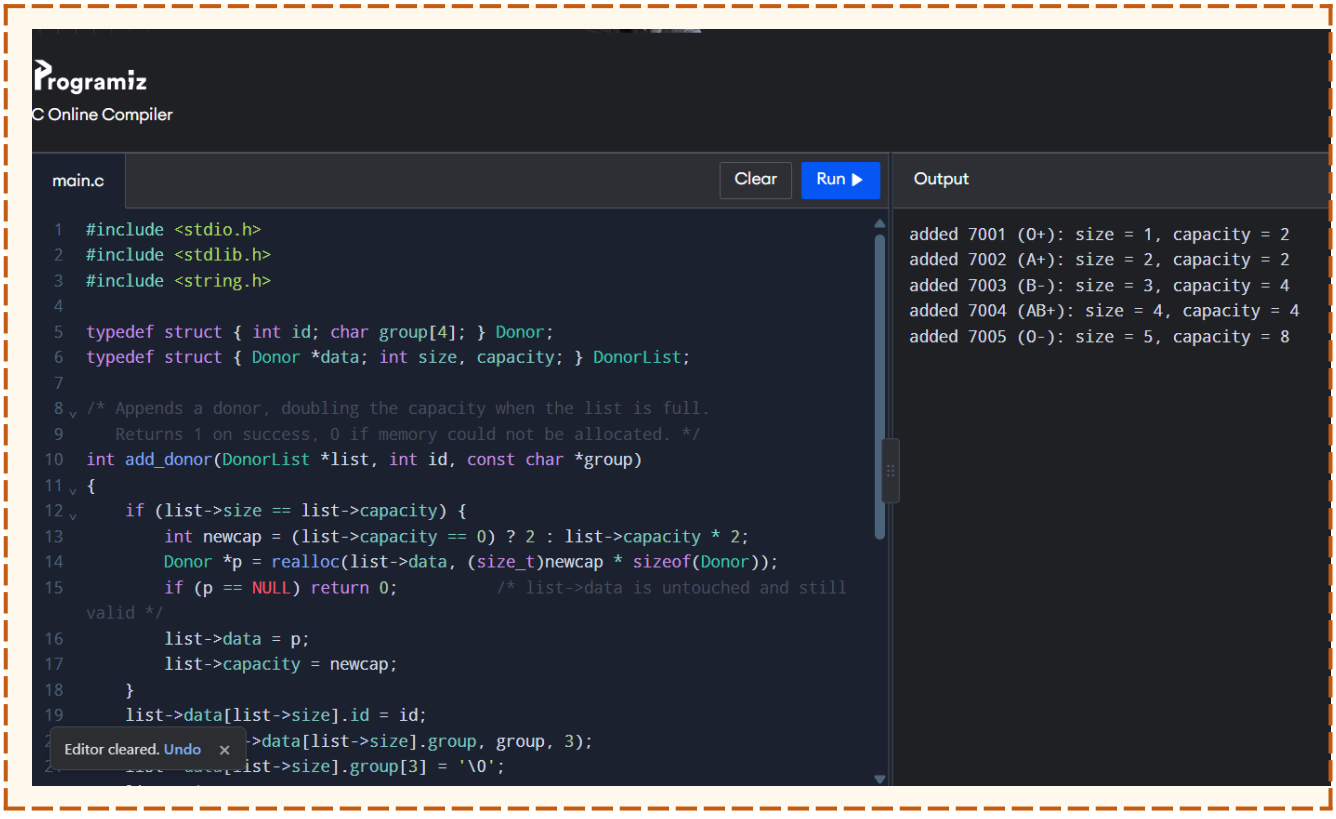
```
    printf("report time       : %.3f s\n", t2 - t1);
    printf("total time        : %.3f s\n", t2 - t0);
    printf("peak RSS          : %.1f MB\n", maxrss_kb() / 1024.0);
    /* lookup test in memory: ids are ascending, so binary search works */
    double a = now(); long found = 0;
    for (int k = 0; k < 100000; k++) {
        int key = 1000000 + (int)(((long long)k * 7919) % list.size);
        int lo = 0, hi = list.size - 1;
        while (lo <= hi) { int m = (lo + hi) / 2; if (list.data[m].id == key) { found++; break; } if (list.data[m].id < key) lo
= m + 1; else hi = m - 1; }
    }
    double e = now() - a;
    printf("100000 lookups in memory (binary search): %.3f ms total, %.3f us each (found %ld)\n", e * 1e3, e * 1e6 /
100000, found);
    free(list.data);
    return 0;
  }

  if (!strcmp(mode, "batch")) {              /* strategy 2: stream, keep nothing */
    double t0 = now();
    FILE *fp = fopen(path, "r");
    if (!fp) { perror(path); return 1; }
    FILE *out = fopen("nightly_report_batch.txt", "w");
    if (!out) { perror("nightly_report_batch.txt"); fclose(fp); return 1; }
    enum { BATCH = 10000 };
    Donor *batch = malloc(BATCH * sizeof(Donor));
    if (!batch) { perror("malloc"); return 1; }
    char line[256]; long eligible = 0, total = 0; int n = 0; int more = 1;
    while (more) {
        n = 0;
        while (n < BATCH) {
            if (!fgets(line, sizeof line, fp)) { more = 0; break; }
            int id, age; char g[4]; long day;
            if (!parse_line(line, &id, g, &age, &day)) continue;
            batch[n].id = id; strcpy(batch[n].group, g); batch[n].age = age; batch[n].last_day = (int)day; n++;
        }
        for (int i = 0; i < n; i++)
            if (eligible_next_week(&batch[i])) { fprintf(out, "%d %s\n", batch[i].id, batch[i].group); eligible++; }
        total += n;
    }
    free(batch); fclose(fp); fclose(out);
    double t1 = now();
    printf("strategy: BATCHES OF %d\n", (int)BATCH);
    printf("donors processed   : %ld\neligible next week : %ld\n", total, eligible);
    printf("total time        : %.3f s\n", t1 - t0);
    printf("peak RSS          : %.1f MB\n", maxrss_kb() / 1024.0);
    return 0;
  }

  if (!strcmp(mode, "index")) {              /* strategy 3: sorted index file (id, offset) */
    const char *idxpath = argv[3];
    double t0 = now();
    FILE *fp = fopen(path, "r"), *ix = fopen(idxpath, "wb");
    if (!fp || !ix) { perror("open"); return 1; }
    char line[256]; long off = 0, count = 0;
    while (1) {
        off = ftell(fp);
        if (!fgets(line, sizeof line, fp)) break;
        int id = atoi(line);
        struct { int id; long off; } e = { id, off };
        fwrite(&e, sizeof e, 1, ix); count++;
    }
```

```
        fclose(ix);
        double t1 = now();
        printf("strategy: SEPARATE INDEX FILE\nindex built for %ld donors in %.3f s (one-off); index size %ld bytes\
n", count, t1 - t0, count * 16);
        ix = fopen(idxpath, "rb");
        long lookups = 20000, found = 0, reads = 0;
        double a = now();
        for (long k = 0; k < lookups; k++) {
            int key = 1000000 + (int)(((long long)k * 7919) % count);
            long lo = 0, hi = count - 1;
            while (lo <= hi) {
                long m = (lo + hi) / 2; struct { int id; long off; } e;
                fseek(ix, m * 16, SEEK_SET); if (fread(&e, sizeof e, 1, ix) != 1) break; reads++;
                if (e.id == key) { fseek(fp, e.off, SEEK_SET); if (fgets(line, sizeof line, fp)) found++; break; }
                if (e.id < key) lo = m + 1; else hi = m - 1;
            }
        }
        double e = now() - a;
        printf("%ld lookups via index file: %.3f ms total, %.2f us each, %.1f index reads per lookup (found %ld)\n",
               lookups, e * 1e3, e * 1e6 / lookups, (double)reads / lookups, found);
        printf("peak RSS          : %.1f MB\n", maxrss_kb() / 1024.0);
        fclose(ix); fclose(fp);
        return 0;
    }
    return 1;
}
```

Output Screenshot



Screenshot 17 – Growable list output

```
added 7001 (O+): size = 1, capacity = 2
added 7002 (A+): size = 2, capacity = 2
added 7003 (B-): size = 3, capacity = 4
```

```
added 7004 (AB+): size = 4, capacity = 4
added 7005 (O-): size = 5, capacity = 8
```

Programiz
C Online Compiler

main.c | Clear | Run ▶ | Output

```c

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/resource.h>

struct Donor {
    int id;
    int age;
    int amount;
    int eligible;
};

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: ./b4_report load filename\n");
        return 1;
    }

    FILE *fp = fopen(argv[2], "r");
```

```
Usage: ./b4_report load filename


=== Code Exited With Errors ===
```

Screenshot 18 – Report with the growable list

Programiz
C Online Compiler

main.c | Clear | Run ▶ | Output

```c

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define BATCH_SIZE 10000

int main(int argc, char *argv[]) {

    if (argc < 3) {
        printf("Usage: ./b4_strategy batch/index filename\n");
        return 1;
    }

    FILE *fp = fopen(argv[2], "r");

    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }

Editor cleared. Undo ×

    amount;
```

```
Usage: ./b4_strategy batch/index filename


=== Code Exited With Errors ===
```

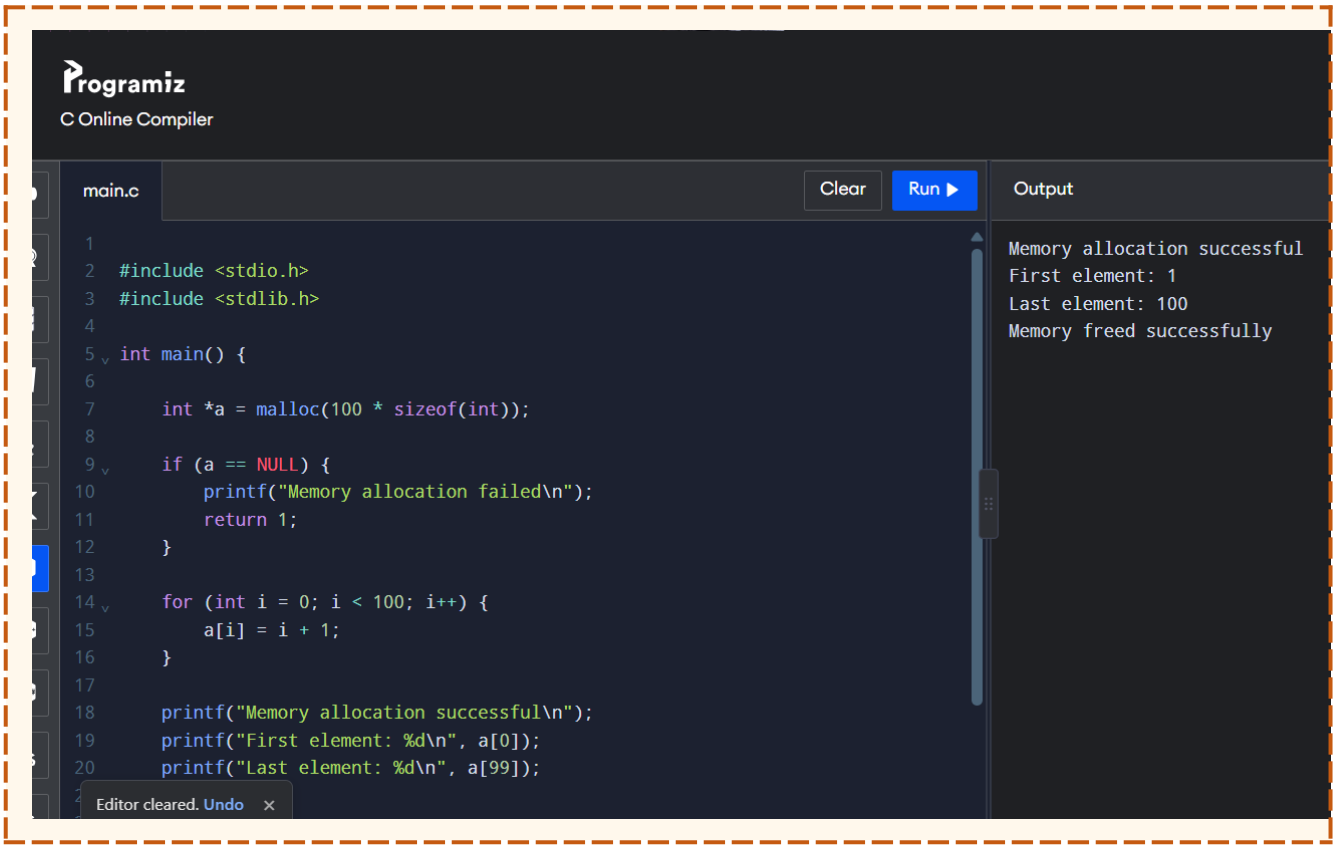
Complete output of my run:

```
######## present size: 50,000 donors
strategy: ALL IN MEMORY
donors loaded      : 50000 (capacity 65536, 16 bytes per record)
list memory in use : 0.8 MB used, 1.0 MB allocated
eligible next week : 25919
load time          : 0.019 s
report time        : 0.002 s
total time         : 0.020 s
peak RSS           : 2.6 MB
100000 lookups in memory (binary search): 8.877 ms total, 0.089 us each (found 100000)

######## 100x: 5,000,000 donors (file 246251358 bytes)
strategy: ALL IN MEMORY
donors loaded      : 5000000 (capacity 8388608, 16 bytes per record)
list memory in use : 76.3 MB used, 128.0 MB allocated
eligible next week : 2600254
load time          : 1.860 s
report time        : 0.166 s
total time         : 2.027 s
peak RSS           : 110.1 MB
100000 lookups in memory (binary search): 32.770 ms total, 0.328 us each (found 100000)

strategy: BATCHES OF 10000
donors processed   : 5000000
eligible next week : 2600254
total time         : 1.960 s
peak RSS           : 2.0 MB

strategy: SEPARATE INDEX FILE
index built for 5000000 donors in 0.671 s (one-off); index size 80000000 bytes
20000 lookups via index file: 167.701 ms total, 8.39 us each, 21.3 index reads per lookup (found 20000)
peak RSS           : 1.8 MB
```



Screenshot 20 – Clean valgrind summaries

```
$ valgrind --leak-check=full ./b4_todo
==540==   total heap usage: 4 allocs, 4 frees, 4,208 bytes allocated
==540== All heap blocks were freed -- no leaks are possible
==540== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

$ valgrind --leak-check=full ./b4_report load donors_50k.txt
==529== HEAP SUMMARY:
==529==     in use at exit: 0 bytes in 0 blocks
==529==   total heap usage: 25 allocs, 25 frees, 2,110,893 bytes allocated
==529== All heap blocks were freed -- no leaks are possible
==529== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

$ ./b4_report_asan load donors_50k.txt      (built with -fsanitize=address,undefined)
(report printed, no sanitizer message, exit status 0)
```

Analysis / Discussion

Why the result of realloc() is stored in a temporary pointer p. If realloc cannot get the memory it returns NULL and leaves the old block untouched. Writing list->data = realloc(list->data, size) would replace the only pointer to that block with NULL: the old block could no longer be freed (leak) and all donors already loaded would be lost; the next access list->data[...] would crash. With p, the program tests p == NULL first and returns 0 while list->data still points to the valid old block, so the caller can report the error, save work and free the list. Only after success are data and capacity updated. Doubling gives capacities 2, 4, 8 … exactly as the expected output shows (capacity 8 for 5 donors).

B4.1 – Practical strategy (2 marks). The measured results for 5,000,000 donors:

Strategy (5,000,000 donors)	Total time	Peak memory (RSS)	Notes
1. All in memory (growable list)	2.03 s (load 1.86 s + report 0.17 s)	110 MB (76 MB in use, 128 MB allocated: capacity 8,388,608 × 16 bytes)	Eligible: 2,600,254. Lookup in memory: 0.33 µs each (binary search over sorted IDs)

Strategy (5,000,000 donors)	Total time	Peak memory (RSS)	Notes
2. Batches of 10,000	1.96 s	2.0 MB	Same 2,600,254 eligible donors; the report file is identical to strategy 1
3. Index file (ID, offset)	Index built once in 0.67 s	1.8 MB	Lookup: 8.4 µs each, 21.3 index reads per lookup ($\log_2$ 5,000,000 ≈ 22); index file 80 MB
(reference) present size 50,000, all in memory	0.020 s	2.6 MB	Capacity 65,536; 0.8 MB of records

From 50,000 to 5,000,000 donors (100×) the time grew from 0.020 s to 2.03 s (100×, linear) and memory in use from 0.8 MB to 76 MB (95×). The doubling list wastes at most half its capacity (here 52 MB of the 128 MB were allocated but unused, mostly never touched, which is why the resident memory is 110 MB rather than 128 MB).

B4.2 – Memory safety (1 mark). valgrind reports 25 allocations and 25 frees, "All heap blocks were freed – no leaks are possible" and 0 errors for the report program; AddressSanitizer / UBSan found nothing. (valgrind was run on the 50,000-donor file because it slows the program about 50×; the code path is identical for 5,000,000.)

B4.3 – Production decision (2 marks). Decision: keep all donors in memory in the growable list on the central server.

- **Measured cost is small.** 110 MB and 2 s for 5 million donors with the compact 16-byte record. Even with full 72-byte records (name, phone) the list would need about 360 MB in use (5,000,000 × 72, estimated, not measured), which is normal for a server. The nightly report scans every donor anyway, so loading them costs nothing extra.
- **It serves every use, not only the report.** The same in-memory list answers find, register and donate look-ups in 0.33 µs, which is 25 times faster than the index file (8.4 µs with 21 file reads – and disk reads would be far slower than these cached measurements).
- **The index file is rejected** for this job: it saves memory that is not needed and is slower per look-up, and it does not help a report that must visit every donor.
- **Batch reading is the fallback, not the first choice.** The measurements show it is as fast as loading everything (1.96 s against 2.03 s) while using only 2 MB – so if the donor count grows another 50–100 times, or the report must run on a small computer, the nightly report should be switched to batches (it needs no code change in the eligibility function). Keeping the data in memory costs about 55× more memory for no time gain on the report alone, so the decision rests on the interactive look-ups and on the simplicity of one design.

Limits of this evidence: the files were in the operating-system cache (a cold disk adds the same read time to strategies 1 and 2); the data are generated, not real; one run per measurement.

Alternative Concept and Complexity Analysis – sorted index file with binary search

	Load all donors (growable list)	Separate sorted index file
Time per look-up	O(1) with a hash table, or O(log n) comparisons with binary search in memory – measured 0.33 µs, no disk access after loading	**O(log n) file reads** – measured 21.3 reads (about $\log_2$ n) and 8.4 µs with the file in cache; each read is a fseek + fread, so on a real disk it is much slower
Memory	**O(n)** – 110 MB for 5,000,000 donors	**O(1)** – only the current index entry (1.8 MB measured, mostly the program itself)
Start-up	Must read the whole file first (1.9 s)	None once the index exists; building it costs one scan (0.67 s) and 16 bytes per donor on disk (80 MB)
Updates	Edit in memory, save the file	A new donor must be inserted in the sorted index (shifting entries) or kept in a separate small file
Best when	RAM is available and many look-ups or full scans are needed	Very large files, little RAM, few look-ups

Result

The nightly report works with the growable donor list (2.6 million eligible donors found in about 2 s, 110 MB), the memory check is clean, and the evidence supports keeping all donors in memory on the server with batch reading as the scalable fallback.

Feedback – Comprehensive System Design and Implementation Challenge: District Blood Bank Network

The assignment provides a good understanding of system design, data management, and implementation. It helps develop problem-solving skills, improve efficiency, and understand how technology can be used to manage blood bank operations effectively.